



“IP地址”应知应懂

北京理工大学计算机学院
金旭亮

网络地址



由于连接在网上的计算机不止一台，不管是采用哪种网络协议，完成数据交换的前提都是为参与通讯的计算机给与唯一确定的网络地址。



计算机中的每个网卡，都拥有一个地址，称为“**MAC地址**”，又称为“物理”地址。



当前计算机网络中，TCP/IP是应用最广泛的通讯协议，相应地，在这个协议族中给计算机所分配的地址，称为“**IP地址**”。



依据地址位数的不同，分为两种IP地址：IPv4和IPv6。



为了方便记忆，人们为IP地址取了“别名”，称为“**域名**”。

查看本机地址信息 (windows)

```
Windows PowerShell
安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS C:\Users\JinXuLiang> ipconfig /all

Windows IP 配置

    主机名 . . . . . : DESKTOP-DQ7I8UF
    主 DNS 后缀 . . . . . :
    节点类型 . . . . . : 混合
    IP 路由已启用 . . . . . : 否
    WINS 代理已启用 . . . . . : 否

以太网适配器 以太网 3:

    媒体状态 . . . . . : 媒体已断开连接
    连接特定的 DNS 后缀 . . . . . :
    描述. . . . . : Realtek USB GbE Family Controller #2
    物理地址. . . . . : 00-E0-4C-68-31-04
    DHCP 已启用 . . . . . : 是
    自动配置已启用. . . . . : 是

以太网适配器 以太网:

    连接特定的 DNS 后缀 . . . . . :
    描述. . . . . : Realtek PCIe GbE Family Controller
    物理地址. . . . . : 1C-69-7A-B8-35-CD
    DHCP 已启用 . . . . . : 是
    自动配置已启用. . . . . : 是
    本地链接 IPv6 地址. . . . . : fe80::45d2:db91:cdda:4b44%20(首选)
    IPv4 地址 . . . . . : 192.168.0.109(首选)
    子网掩码 . . . . . : 255.255.255.0
    获得租约的时间 . . . . . : 2022年5月21日 8:53:40
    租约过期的时间 . . . . . : 2022年5月21日 18:01:29
    默认网关 . . . . . : 192.168.0.1
    DHCP 服务器 . . . . . : 192.168.0.1
    DHCPv6 IAID . . . . . : 136079738
    DHCPv6 客户端 DUID . . . . . : 00-01-00-01-28-50-A9-55-1C-69-7A-B8-35-C
```

MAC地址由6个字节组成，前3个为厂商代码，后3个为厂商给出的唯一标识。

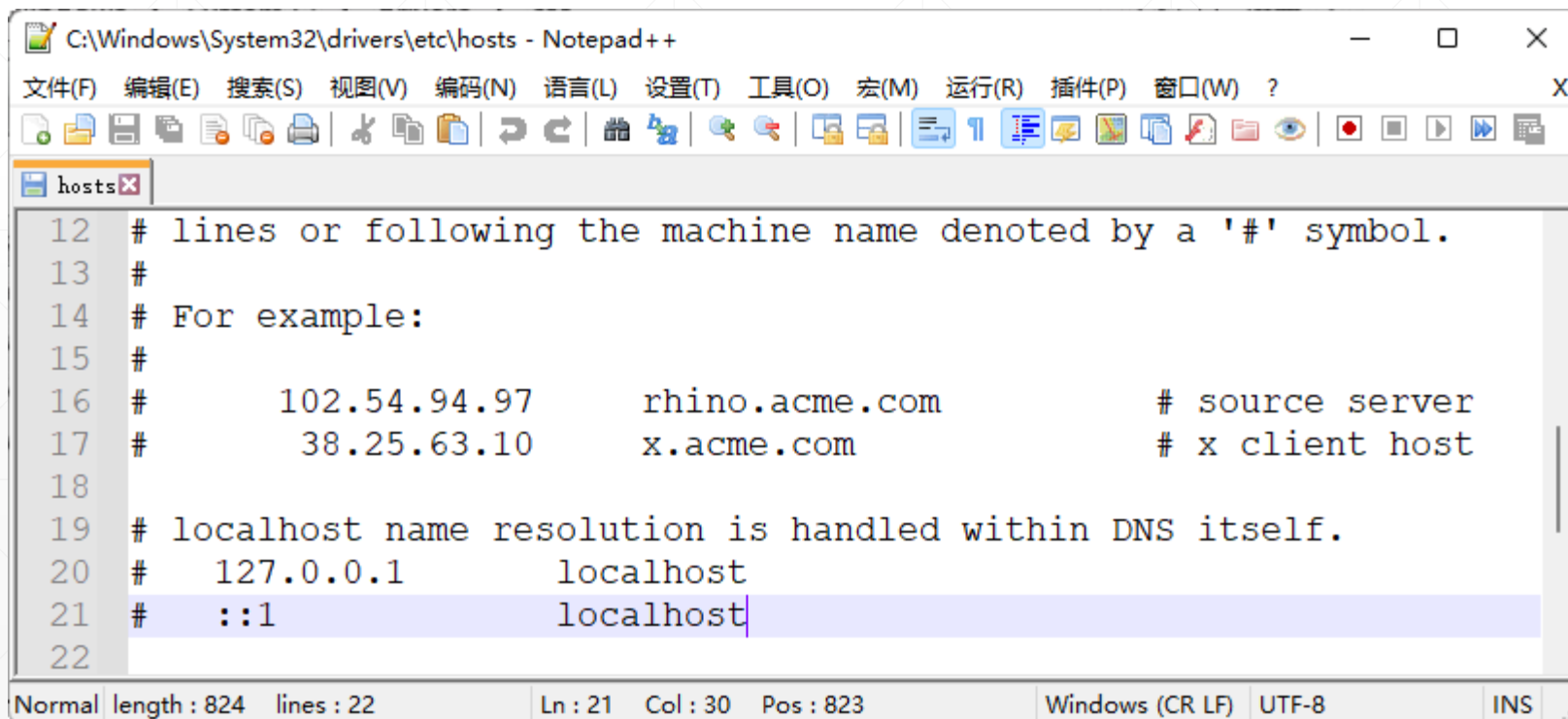
localhost vs. 127.0.0.1



localhost只是一个域名，只有把域名localhost解析为127.0.0.1，才能进行数据传输与通信。



解析的过程是由hosts文件完成的，该文件内容如下：



```
C:\Windows\System32\drivers\etc\hosts - Notepad++
文件(F) 编辑(E) 搜索(S) 视图(V) 编码(N) 语言(L) 设置(T) 工具(O) 宏(M) 运行(R) 插件(P) 窗口(W) ?
hosts
12 # lines or following the machine name denoted by a '#' symbol.
13 #
14 # For example:
15 #
16 #      102.54.94.97      rhino.acme.com      # source server
17 #      38.25.63.10      x.acme.com          # x client host
18
19 # localhost name resolution is handled within DNS itself.
20 #   127.0.0.1      localhost
21 #   ::1            localhost
22

Normal length : 824 lines : 22 Ln : 21 Col : 30 Pos : 823 Windows (CR LF) UTF-8 INS
```

新问题出现.....



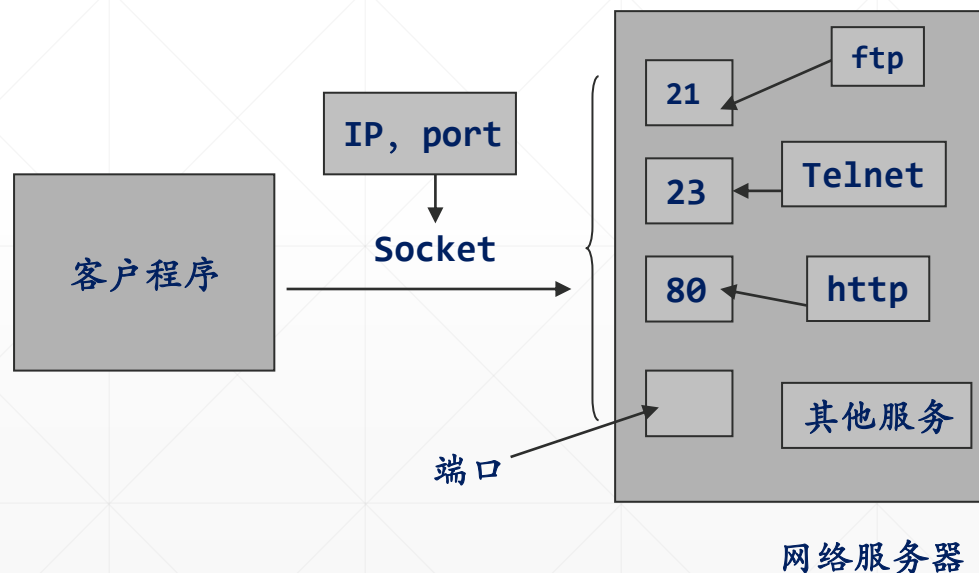
有了IP地址，我们就可以区分开所有连在网络上的计算机，但新问题出现了：

一台网络计算机上可能跑着多个网络应用程序，它们可能会使用同一个网络接口从网络中接收（或发送）数据，因而共享同一个IP地址，在这种情况下，你怎么将到达主机的数据包转发给真正的“需求者”？

端口

为了解决这个问题，TCP/IP协议设计者引入了“**端口 (Port)**”这个概念，规定每个提供网络服务的应用程序都必须指定一个“端口”，不同的网络应用程序不能使用相同的端口。

IP+端口号，可以完全区分出每台网络计算机上运行的所有网络程序。



端口号的表示

端口号是用整数来表达的，其范围为 $[0, 65535]$ ，其中 $[0, 1023]$ 为系统所保留，专门提供给那些通用的服务（well-known services）使用，如http服务的端口号为80，ftp服务的端口为21，等等。因此，当我们编写通信程序时，应选择一个大于1023的数作为端口号，以免发生冲突。

那么，特定网络应用程序的IP地址和端口号是怎样在网络中传播的？

TCP数据包中的端口

TCP Header																																
Bit offset	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	Source port																Destination port															
32	Sequence number																															
64	Acknowledgment number																															
96	Data offset				Reserved				C W R	E C E	U R G	A C K	P S H	R S T	S Y N	F I N	Window Size															
128	Checksum																Urgent pointer															
160	Options (if Data Offset > 5)																															
...	...																															



每个TCP数据包都包容着两个端口信息（每个端口占2个字节，是一个16位的二进制数值，所以，最大端口值为2的16次方减1，为65535）。“源端口号（Source port）”指明发送此数据包的网络应用程序使用的端口，“目标端口号（Destination port）”指明接收方网络应用程序使用的端口。



这样一来，接收方计算机依据目标端口号，就可以把此数据包转发给真正的网络应用程序。

IP数据包中的IP地址信息

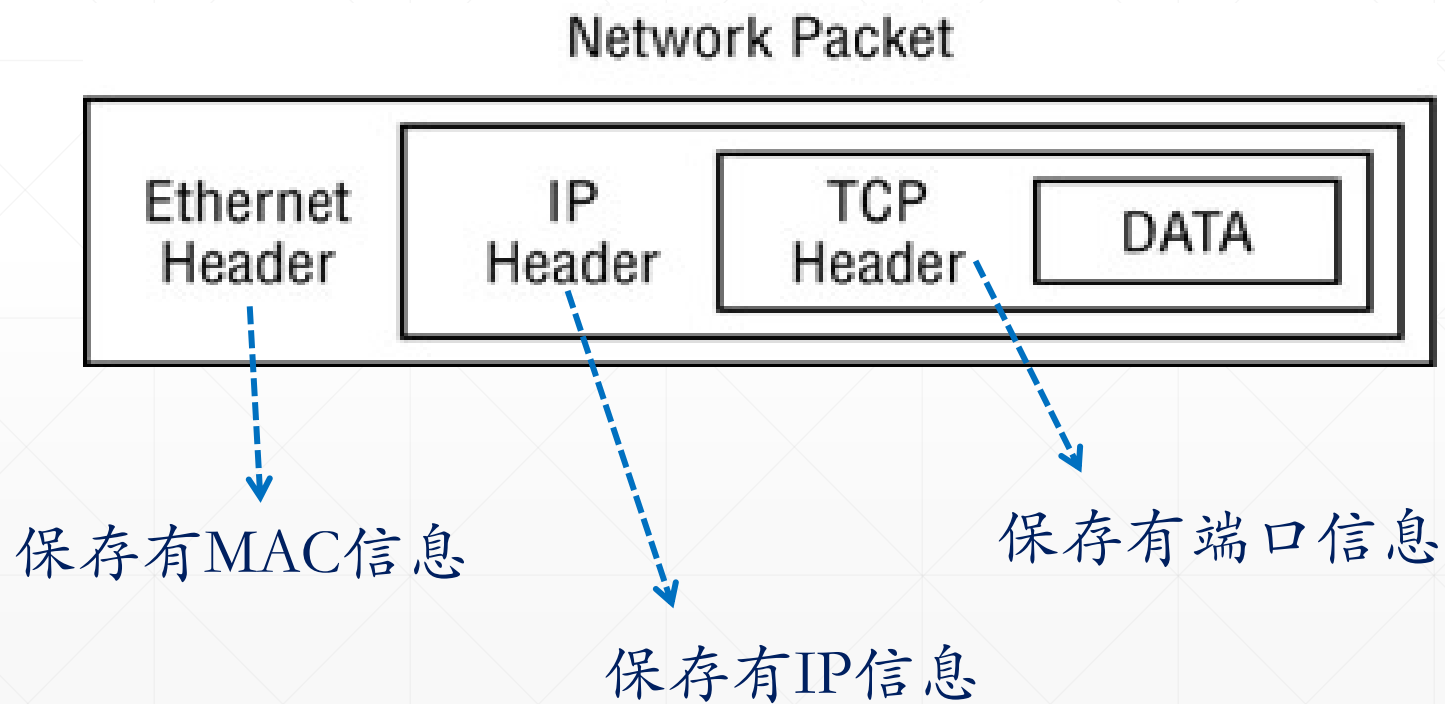
bit offset	0 - 3	4 - 7	8 - 13	14-15	16 - 18	19 - 31
0	Version	Header Length	<u>Differentiated Services Code Point</u>	<u>Explicit Congestion Notification</u>	Total Length	
32	Identification				Flags	Fragment Offset
64	Time to Live	<u>Protocol</u>			Header Checksum	
96	<u>Source IP Address</u>					
128	<u>Destination IP Address</u>					
160	Options (if Header Length > 5)					
160 or 192+	Data					



承载TCP数据包信息

Packet的结构

在网络上传送的数据包是嵌套的：



IP终结点是网络服务的访问点



IP地址与端口唯一地标识了一个网络中的网络应用程序，我们将这个组合称为“**IP终结点 (IP EndPoint)**”，它是一个网络服务的访问点。



在Java中，可以使用InetAddress对象来代表一个IP地址，出于方便考虑，这个类也能直接使用“域名”代替IP。



网络开发中，使用Socket来封装IP地址及端口信息，因此，我们通常将“经典”的直接基于TCP/UDP的开发工作称为“**Socket编程**”。